condition occurs. But loop is also smart when you use an Enumerator—when an enumerator object runs out of values inside a loop, the loop will terminate cleanly.

The following example shows this in action—the loop iterates both arrays in parallel and ends when the three-element enumerator runs out of values. You can also handle this in your own iterator methods by rescuing the StopIteration exception, but because we haven't talked about exceptions yet, we won't go into details here.

```
short_enum = [1, 2, 3].to_enum
long_enum  = ('a'..'z').to_enum

loop do
  puts "#{short_enum.next} - #{long_enum.next}"
end
```

produces:

```
1 - a
2 - b
3 - c
```

Enumerators Are Objects

Enumerators take something that's normally executable code (the act of iterating—by default, calling each) and turn it into an object. This means you can do things programmatically with enumerators that aren't easily done with regular loops.

For example, the Enumerable module defines each_with_index. This invokes its host class's each method, returning successive values along with an index:

```
result = []
['a', 'b', 'c'].each_with_index { |item, index| result << [item, index] }
result    # => [["a", 0], ["b", 1], ["c", 2]]
```

But what if you wanted to iterate and receive an index but use a different method than each to control that iteration? For example, you might want to iterate over the characters in a string. There's no method called each_char_with_index built into the String class.

Enumerators to the rescue. The each_char method of strings will return an enumerator if you don't give it a block, and you can then call each_with_index on that enumerator:

```
result = []
"cat".each_char.each_with_index { |item, index| result << [item,  index] }
result    # => [["c", 0], ["a", 1], ["t", 2]]
```

In fact, this is such a common use of enumerators that Matz has given us with_index, which makes the code read better:

```
result = []
"cat".each_char.with_index { |item, index| result << [item,  index] }
result    # => [["c", 0], ["a", 1], ["t", 2]]
```

By separating the with_index from the each_char, we can even chain in a map call and simplify the code even further:

```
"cat".each_char.with_index.map { |item, index| [item,  index] }
```

You can also create the Enumerator object explicitly—in this case we'll create one that calls our string's each_char method. We can call to_a on that enumerator to iterate over it: